DAY 08 · THEORY HANDOUT

Neural Networks: From Inputs to Predictions

Inputs, weights, bias, loss, and the steps that turn examples into a learned model.

KGP Talkie

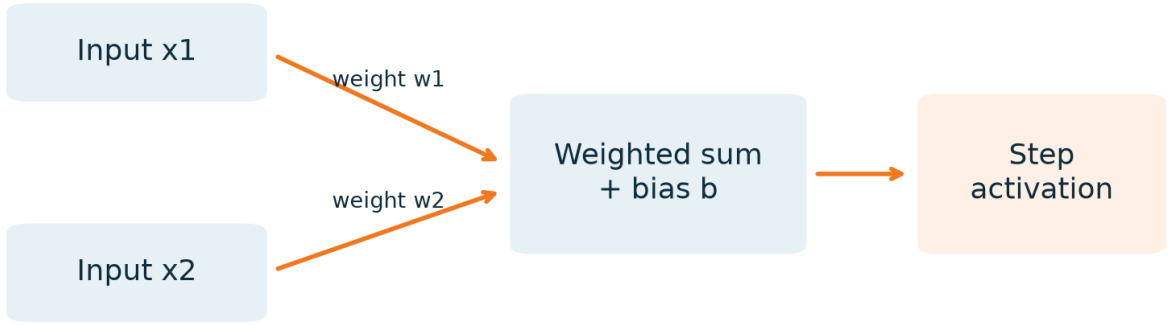
Scan to subscribe

Laxmi Kant Tiwari · kgptalkie.com

Perceptron: one small decision unit

A neuron is a small calculation. A classic perceptron uses it to choose a class.

Perceptron: inputs, weights, bias, sum, and activation



Score z = x1*w1 + x2*w2 + b. The step outputs class 0 or class 1.

Inputs are the feature values. Each has a **weight**, a number that multiplies it. A negative weight can reduce the score.

Bias is an extra number added to the weighted sum. It shifts the cut point without needing another measured feature.

An **activation function** turns the score into an output. The perceptron uses a step: class 0 or class 1.

Weights and bias are model parameters. Training changes them using known answers.

Two rows and two columns

First use a number output so that we can see the weight and bias calculation.

Input x	Target y
10	25
5	15

Two rows and two columns. The second column is the target, not a second input.

x is the input. **y** is its known target. When x is 10, y is 25. When x is 5, y is 15.

A **linear neuron** keeps its number output: prediction = weight x input + bias. It does not apply the perceptron class step.

Start with candidate values: **weight = 1**, **bias = 1**. These are guesses before fitting.

> One input and one target make two columns. Two feature columns would be a different example.

04 KGP Talkie

Make a prediction and measure its error

The first candidate predicts 11 for input 10, and 6 for input 5.

Actual	Prediction	Error	Squared error
25	11	14	196
15	6	9	81

Mean Squared Error = (196 + 81) / 2 = 138.5

Error is actual minus predicted. The first error is 25 - 11 = 14. The second is 15 - 6 = 9.

Loss is one number used to judge prediction mistakes during training. Here we use **Mean Squared Error (MSE)**.

Square each error, add them, then divide by the number of records. Lower loss means closer predictions on these records.

A zero training loss on two records does not prove that a model will predict new records well.

Change the weight and bias

Compare each candidate using the same two known records.

w	b	Prediction for 10	Prediction for 5	MSE
1	1	11	6	138.5
2	2	22	12	9
3	3	33	18	36.5
2	5	25	15	0

Weight 2 and bias 2 give predictions 22 and 12. Their errors are 3 and 3, so MSE is 9.

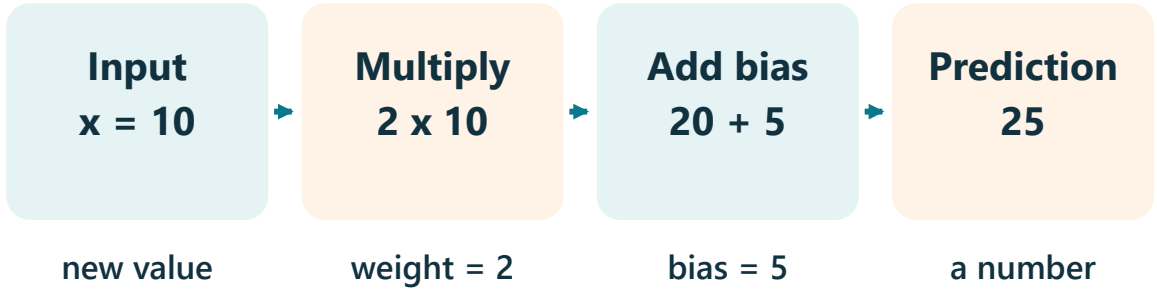
Weight 3 and bias 3 give 33 and 18. Their squared errors are 64 and 9. MSE rises to 36.5.

Weight 2 and bias 5 give 25 and 15. Both predictions now match the known targets.

> Bigger parameter values do not always give smaller loss.

Use the same rule on a new input

The fitted number rule is prediction = 2 x input + 5.



For a new input of **7**, the same rule gives 2 x 7 + 5 = **19**. The diagram traces input 10 through the same calculation.

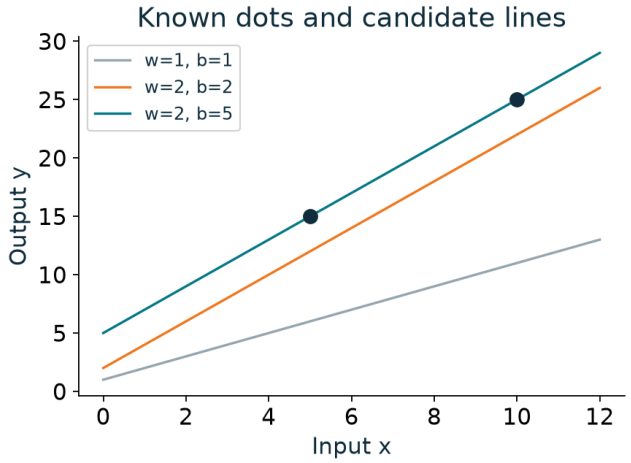
This step is **prediction**, also called inference. It uses the saved values. It does not adjust the weight or bias.

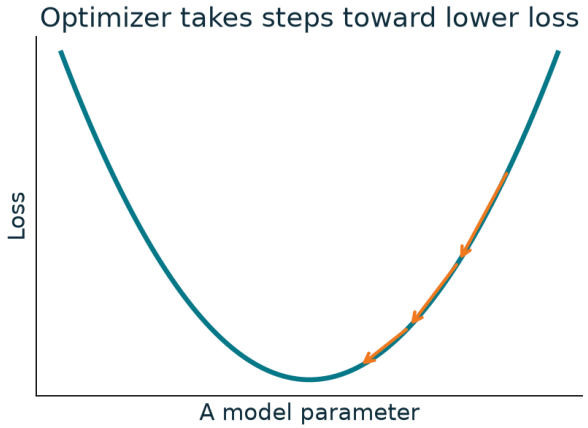
The target for a new input may be unknown. The output is an estimate until we can compare it with a real answer.

> Training learns values. Prediction uses those values.

An optimizer takes learning steps

The training method uses the loss to guide changes in model parameters.





An **optimizer** is the method that updates weights and bias values. It does not need to try every possible pair.

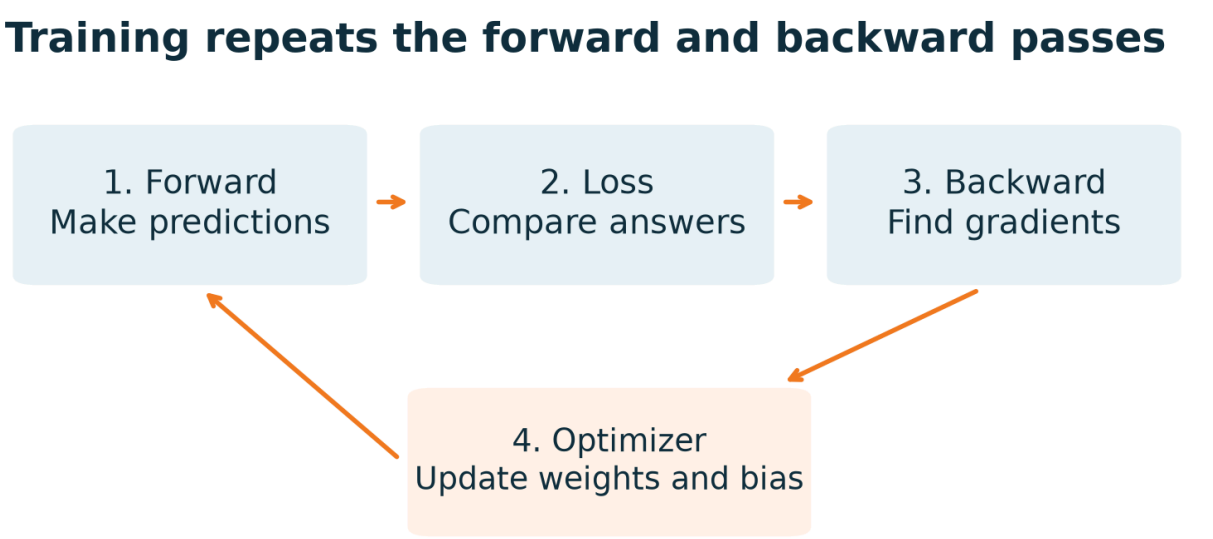
A **gradient** says how a small parameter change would affect the loss. The optimizer uses this direction to seek lower loss.

The **learning rate** controls step size. Very large steps can skip past a useful setting. Very small steps can learn slowly.

> The bowl is a simple picture of the idea. A network with many parameters can have a more complex loss surface.

Forward and backward propagation

A pass moves through the network. Training repeats passes over the examples.



Forward propagation: use the current weights and biases to make predictions. Then compare them with targets to calculate loss.

Backpropagation: calculate gradients from the output back through earlier layers. The optimizer then updates the parameters.

An **epoch** is one pass through the training records. Training stops at a limit or when improvement becomes too small.

Lower training loss helps, but held-out records are needed to check new-data performance.

What if we have two inputs?

Each feature gets its own weight. Their weighted values are added together.

Input x1	Input x2	Weighted sum + bias	Step class
0.4	0.8	0.5 x 0.4 + 0.5 x 0.8 - 0.6 = 0	1
0.2	0.3	0.5 x 0.2 + 0.5 x 0.3 - 0.6 = -0.35	0

Each row uses the same weights. Our step rule gives 1 when the score is 0 or more.

Here X has two records and two feature columns. The target classes are separate: **y = [1, 0]**.

The chosen weights are 0.5 and 0.5. The bias is -0.6. Multiply both inputs, add them, then add the bias.

The step rule gives class 1 for the first record and class 0 for the second. A step output is a class, not a probability.

With more inputs, add one weighted term for each input. The order of features must stay the same.

A perceptron learns from a wrong class

For this update, begin with both weights at 0.5 and bias at -0.2.

Parameter	Before	Change	After
Weight for x1	0.5	0.1 x (-1) x 0.2 = -0.02	0.48
Weight for x2	0.5	0.1 x (-1) x 0.3 = -0.03	0.47
Bias	-0.2	0.1 x (-1) = -0.1	-0.3

Learning rate = 0.1. Actual class = 0. Predicted class = 1. Error = -1.

For the second record [0.2, 0.3], the old score is 0.05. The step predicts 1, but its known class is 0. Error = 0 - 1 = **-1**.

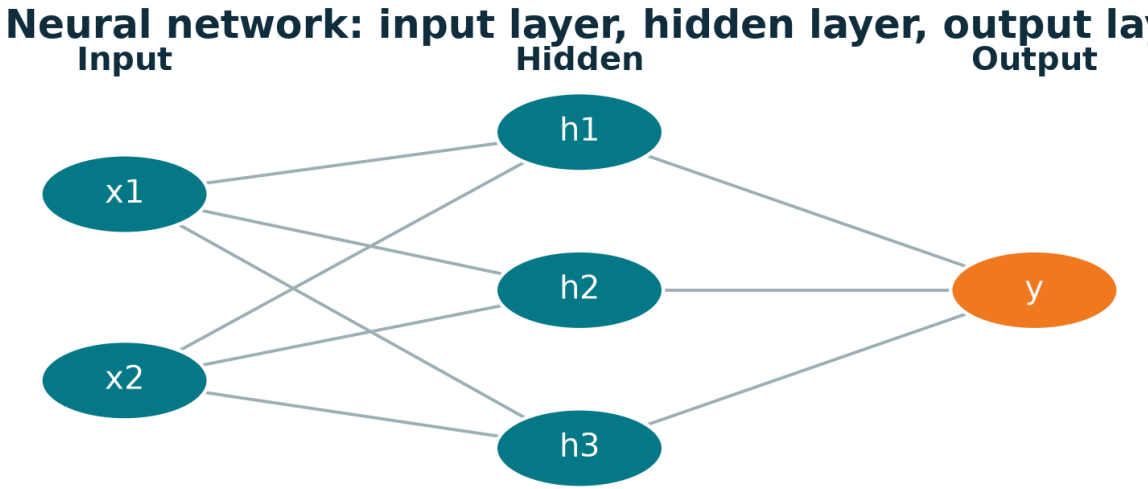
The **perceptron learning rule** adds learning rate x error x input to each weight. Bias gets learning rate x error.

After the update, the score is 0.48 x 0.2 + 0.47 x 0.3 - 0.3 = **-0.063**. The step now predicts the correct class 0.

This is the perceptron class update. Multilayer networks use activations whose changes can be measured, so backpropagation can find gradients.

Link neurons into a network

A layer is a group of units at the same stage of the calculation.



Each line has a weight. Each hidden and output neuron has its own bias.

Input layer: receives the measured features. **Hidden layer:** turns them into learned intermediate values. **Output layer:** returns the prediction.

A **fully connected layer** connects each neuron to every unit in the previous layer. Each connection has its own weight.

Each hidden and output neuron also has a bias. Hidden units are not the same thing as biases.

> An Artificial Neural Network (ANN) is inspired by linked brain cells, but these units are number calculations.

Follow a small forward pass

Use the two inputs 0.4 and 0.8. Each hidden neuron makes its own calculation.

Hidden neuron	Weights for x1, x2	Bias	Score	ReLU output
h1	0.5, 0.5	-0.6	0	0
h2	1, 0	0	0.4	0.4
h3	0, 1	-0.5	0.3	0.3

Inputs: x1 = 0.4, x2 = 0.8.
Output score = 1 x 0 + 1 x 0.4 + 1 x 0.3 - 0.5 = 0.2.
Sigmoid(0.2) is about 0.55. A 0.5 threshold gives class 1.

Each hidden score is its weighted inputs plus its own bias. **ReLU** keeps a positive score and changes a negative score to 0.

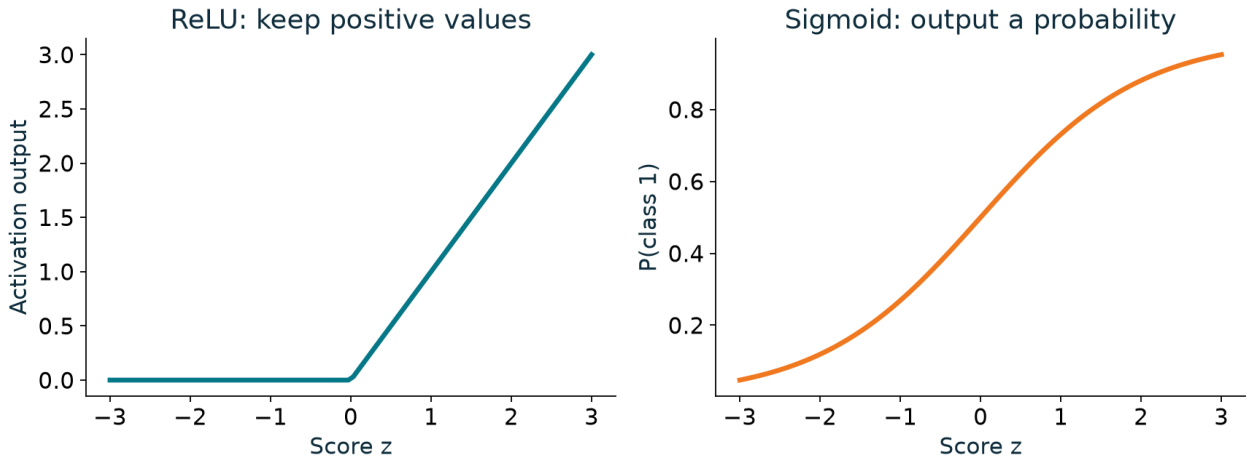
The three outputs are 0, 0.4, and 0.3. The output neuron combines them using its own weights and bias.

Sigmoid turns the final score 0.2 into a probability of about 0.55. A 0.5 threshold gives class 1.

Hidden activations, output weights, and output bias all matter. We cannot just add raw hidden scores in every network.

Why use an activation function?

The choice depends on where the unit is used and what the output should mean.



ReLU: negative scores become 0; positive scores stay as they are. Hidden neurons can then learn patterns beyond a single straight line.

Sigmoid: gives a number between 0 and 1. A binary classifier can use it for the probability of class 1.

A number-prediction output can keep its raw value. Stacking only linear calculations still gives one overall linear calculation.

For classification, cross-entropy or log loss judges probability mistakes. Our first number example used MSE.

More features and ndim

A feature is one input fact. An array dimension is one axis of its layout.

X shape	Records	Features per record	X.ndim
(2, 1)	2	1	2
(2, 2)	2	2	2
(569, 30)	569	30	2

A row with 30 features is a 30-feature vector. X is still a two-dimensional NumPy array.

A record with 30 features has 30 input values. A table of those records still has only rows and columns.

X.ndim = 2 for that table. Its shape is (number of records, number of features). The feature count is not NumPy ndim.

Image pixels can also become numeric inputs. More inputs need more matching weights; the target still stays separate.

> A model needs the same feature meaning and order during training and prediction.

From a table to ANN inputs

The built-in breast cancer dataset has 569 records, 30 features, and two target classes.

Keep the test rows outside the learning steps

569 records -> 398 training records + 171 test records

Train: fit scaler -> train ANN

Test: same scaler -> predict

Each row holds measurements from a cell sample. Class **0 is malignant**, meaning cancerous. Class **1 is benign**, meaning non-cancerous.

About 30% of rows are held out: 398 training records and 171 test records. Matching inputs and labels stay together.

Scaling subtracts each training feature mean and divides by its spread. Test values use the same saved rule.

> This dataset is a coding example, not a tool for diagnosing a person.

The Multilayer Perceptron

An MLP is a network with one or more hidden layers.

Setting	Meaning	Reason in this example
10, 15, 10	Three hidden layers	The network size used in the resource
ReLU	Hidden activation	Lets the model learn patterns beyond one straight line
Adam	Optimizer	Uses gradients to update parameters
0.05	Starting learning rate	The example step size
500	Maximum training passes	A limit; training may stop earlier

The network has **30 input features**, then hidden layers of **10, 15, and 10 neurons**, then a binary probability output.

A **hyperparameter** is a setting chosen before training. A **parameter** is a learned weight or bias.

After the output probability is found, a class rule chooses 0 or 1. Accuracy compares those classes with known targets.

> These settings describe this example. They are not the best settings for every dataset.

Check the held-out predictions

Rows are actual classes. Columns are predicted classes.

Actual / Predicted	0: malignant	1: benign
0: malignant	61	3
1: benign	1	106

171 held-out records. 167 correct predictions. Accuracy = 167 / 171 = 97.66%.

The model gets **167 of 171 records correct**. That is about **97.66% accuracy** for this split.

3 malignant records were predicted benign. **1 benign record was predicted malignant**. The matrix shows what the accuracy number alone hides.

ANNs can learn complex patterns, but they can also overfit, take time to train, and be hard to explain. A good result on one split has limits.

Reading: Machine Learning Algorithms in Practice, Ashok Veda Publications, 2024, Module 17.

DAY 08 · COMPLETE

Thank You

Follow KGP Talkie for new lessons, course updates and live sessions.

Scan to follow

Instagram
@kgptalkie

WhatsApp Channel
KGP Talkie